Linear Equations Solver

A C++ Object-Oriented System for Parsing, Manipulating & Solving Linear Systems

https://github.com/yt29501240201877-cloud/LinearSystem

$$Ax = b$$

Gaussian Elimination | Cramer's Rule | Row Operations | C++ OOP

Project Overview

Purpose

Build a C++ program that reads, stores, and manipulates systems of linear equations – supporting row operations and two solving strategies: Gaussian Elimination and Cramer's Rule.

Scope

Handles n-variable systems entered as text. Supports parsing, printing, column extraction, addition, subtraction, substitution, and matrix-based solutions.

Design

Two-class OOP architecture: Equations holds the data model (coefficients + constant), LinearSys inherits it and provides all operations and solvers.

Interface

Interactive command-line loop. Accepts: num_vars, equation, column, add, subtract, substitute, D, D xN, D_value, solve, quit.

Class Architecture

Equations

Fields

vector<float> coefficients
float con

Constructor

Equations(int vars = 0)
resizes coefficient vector
initialises con = 0

inherits

LinearSys : Equations

Fields

vector<Equations> equations
int numVars

Methods

addEquation() num_vars() printEquation()
column() add() subtract()
substitute() D() D_var() D_value()
solve() — Gaussian & Cramer's

Command Reference

Command	Description	Example
`num_vars`	Print number of variables in the system	`num_vars`
`equation i`	Print equation i with all its coefficients	`equation 2`
`column xi`	Print all coefficients for variable xi	`column x1`
`add a b`	Print the sum of equations a and b	`add 1 2`
`subtract a b`	Print the difference of equations a and b	`subtract 2 3`
`substitute xi a b`	Eliminate xi from equation a using equation b	`substitute x2 1 3`
`D`	Print the Cramer's coefficient matrix	`D`
`D xi`	Cramer's matrix with constants replacing column xi	`D x2`
`D_value`	Print the determinant of the coefficient matrix	`D_value`
`solve`	Solve the system via Gaussian Elimination	`solve`

Equation Parsing — addEquation()

1. **Read input string**
   e.g. "3x1+2x2+1x3=12"

2. **Split on '='**
   left = "3x1+2x2+1x3" right = "12"

3. **atof(right)**
   con = 12.0

4. **Tokenise left on '+'**
   tokens: "3x1", "2x2", "1x3"

5. **Per token: find 'x'**
   coef = atof(before x), index = atoi(after x)

6. **Store in vector**
   coefficients[index-1] = coef

addEquation() — Core Logic

```
int pos = input.find('=');
eq.con = atof(right);

while getline(ss, term, '+') {
  xPos = term.find('x');
  coef  = atof(term[0..xPos]);
  index = atoi(term[xPos+1]);
  coefficients[index-1] = coef;
}
equations.push_back(eq);
```

Row Operations

add(a, b)

```
eq_a[j] + eq_b[j]
```

Adds coefficients and constants from equations a and b element-by-element and prints the resulting equation. Does not modify stored equations.

```
add 1 2  →  3x1+4x2+3x3=26
```

subtract(a, b)

```
eq_a[j] - eq_b[j]
```

Subtracts equation b from equation a term-by-term. Useful for manual Gaussian-style elimination steps.

```
subtract 2 1  →  -1x1+2x2+1x3=8
```

substitute(xV, a, b)

```
ratio = eq_a[xV] / eq_b[xV]
```

Eliminates variable xV from equation a using equation b as the source. Computes ratio then applies: coef[j] = coef_a[j] − ratio × coef_b[j]. Skips zero terms.

```
substitute x2 1 2  →
1.667x1+0.333x3=3.33
```

Gaussian Elimination — solve()

Phase 1 — Build Matrix

- Copy coefficients into n×(n+1) augmented matrix
- Last column stores the constant (con) of each equation

Phase 2 — Forward Elimination

- For each pivot row i: check matrix[i][i] ≠ 0
- For each row j below: ratio = matrix[j][i] / matrix[i][i]
- Subtract ratio × pivot row from row j
- Produces upper-triangular form

Phase 3 — Back Substitution

- Start from last equation (1 unknown)
- Solve upward: solution[i] = (rhs − known terms) / diagonal
- Print x1= x2= x3= etc

Pseudocode

```
// forward elimination
for i in 0..n:
  if matrix[i][i] == 0 → No Solution
  for j in i+1..n:
    ratio = matrix[j][i] / matrix[i][i]
    matrix[j][k] -= ratio * matrix[i][k]

// back substitution
for i in n-1..0:
  sol[i] = matrix[i][numVars]
  for j in i+1..n:
    sol[i] -= matrix[i][j] * sol[j]
  sol[i] /= matrix[i][i]
```

Cramer's Rule — D(), D_var(), D_value()

Cramer's Rule states: xi = det(Ai) / det(A)

where A is the coefficient matrix and Ai replaces column i with the constants vector.

D()

Prints the n×n coefficient matrix surrounded by | … | pipes.
All rows, all variable columns, no constants.

```
D
```

D_var(xi)

Replaces column xi with the constants (con) vector — the Cramer's matrix for solving for variable xi.

```
D x2
```

D_value()

Computes det(A) via recursive cofactor expansion and prints the result. Used to check solvability.

```
D_value  →  D = 13
```

Sample Session

linear_solver — bash

```
$ ./linear
Enter number of equations: 3
Enter equations:
2x1+1x2+1x3=9
1x1+3x2+2x3=17
1x1+2x2+4x3=20

> D_value
D = 13

> substitute x2 1 2
1.66667x1+0.333333x3=3.33333

> solve
x1=1
x2=2
x3=4
```

D_value

det(A) = 13 ≠ 0, so a unique solution exists. System is solvable.

substitute x2 1 2

Eliminates x2 from eq 1 using eq 2 as the source. x2 coefficient becomes 0.

solve

Gaussian Elimination finds x1=1, x2=2, x3=4. Verify: 2(1)+1(2)+1(4)=8≠9… check!

Summary

Linear Equations Solver

- OOP design — Equations data model + LinearSys operations engine cleanly separated
- Row operations: add, subtract, substitute for interactive equation manipulation
- Gaussian Elimination for fast algebraic solution of n-variable systems
- Cramer's Rule support: D(), D_var(), D_value() for determinant-based solving
- Extensible architecture ready for pivoting, file I/O, and additional solvers

Built with C++ · Object-Oriented Design · Gaussian Elimination · Cramer's Rule

Participants

Full Name	Military ID
Youssef Safwat Sobhy	11205
Mohamed Adel Abd ElAl	11194
Omar Mahmoud Ali	11200
Omar Ibrahim El Safey	11199
Fathi Mohamed Fathi	11225
Omar Mohamed Mohamed	11190

$$Ax = b$$

Gaussian Elimination | Cramer's Rule | Row Operations | C++ OOP